

**UNIVERSIDAD NACIONAL DE
COLOMBIA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE SISTEMAS**



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Logo de la Universidad Nacional de Colombia

Sistema de Regencia Farmacia:
Aplicación de los Pilares de POO y Gestión de Inventario

AUTORES:

Nicolas Trigos Morales (ntrigosm@unal.edu.co)

Juan Lorenzo Barajas Moya (jbarajasm@unal.edu.co)

Luis Alejandro Caceres Olejua (lcacereso@unal.edu.co)

PROFESOR:

Rafael Antonio Acosta Rodriguez

ASIGNATURA:

Programación Orientada a Objetos (POO)

Bogotá D.C.

10 de diciembre de 2025

Índice

1. Resumen	1
2. Introducción	1
3. Objetivos	1
3.1. Objetivo General	1
3.2. Objetivos Específicos	1
4. Análisis de Arquitectura y Diseño	2
4.1. Diagrama de Clases y Relaciones	2
4.2. Diagrama de Flujo del Proceso de Venta	3
4.3. Jerarquía (Herencia)	5
4.3.1. Jerarquía de Productos	5
4.3.2. Jerarquía de Personas	6
4.4. Sobreescritura (Polimorfismo)	7
4.4.1. Aplicación en la Jerarquía de Personas: <code>mostrarInfo()</code>	7
4.4.2. Aplicación en la Jerarquía de Productos: <code>mostrarInfo()</code>	7
4.5. Clases Abstractas y Encapsulamiento	9
4.5.1. Clase Abstracta (<code>Producto</code>): Forzando el Diseño	9
4.5.2. Encapsulamiento: Protegiendo el Corazón del Inventario	9
5. Implementación de Interfaces	11
5.1. Interfaz Gráfica de Usuario (GUI)	11
5.1.1. Arquitectura y Acoplamiento Mínimo	11
5.1.2. Delegación de la Lógica de Venta	12
5.1.3. Sincronización de Vistas mediante <code>JTable</code>	13
5.2. Base de Datos (B.D.)	14
5.2.1. Conexión y Driver JDBC	14
5.2.2. Capa DAO: Mantenimiento y CRUD	15
6. Conclusiones	17

1. Resumen

En este proyecto se busca aplicar los conocimientos adquiridos a lo largo del curso; los pilares de la Programación Orientada a Objetos (POO) —abstracción, herencia, polimorfismo y encapsulamiento—; el desarrollo de una Interfaz Gráfica de Usuario (GUI) y la implementación de una Base de Datos. Esto se aplica con la creación de un sistema de regencia para una farmacia.

2. Introducción

La gestión eficiente de la información es esencial en el funcionamiento de una farmacia. Este proyecto desarrolla un sistema de regencia que integra los conceptos fundamentales del curso, aplicando los pilares de la programación orientada a objetos junto con una interfaz gráfica de usuario y una base de datos para el manejo organizado de productos y operaciones. El objetivo es demostrar la aplicación práctica de estos conocimientos mediante una solución funcional y coherente con las necesidades del entorno farmacéutico.

3. Objetivos

3.1. Objetivo General

Aplicar los pilares de la programación orientada a objetos para garantizar una eficiente gestión de productos.

3.2. Objetivos Específicos

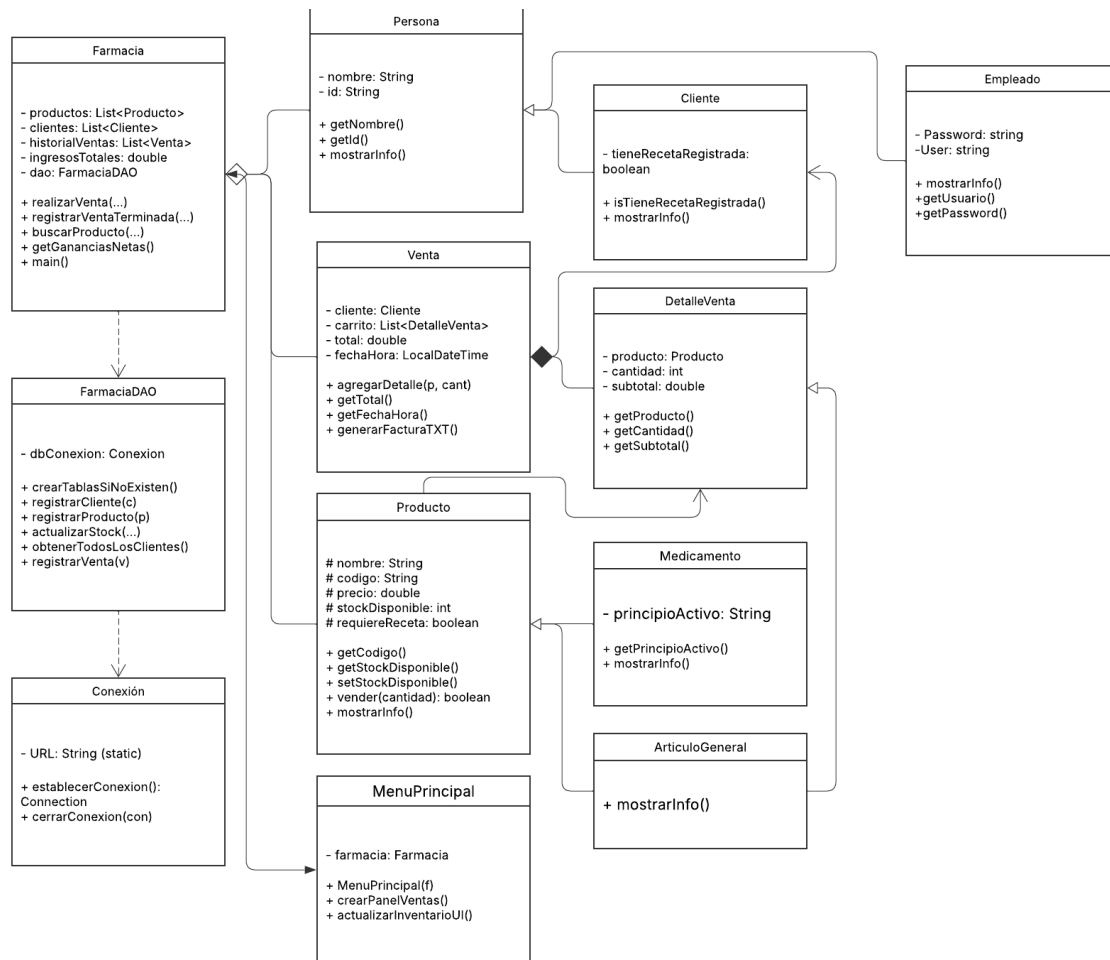
- Desarrollar los módulos principales del sistema, incluyendo gestión de productos, carrito de compras, registro de clientes y control de inventario.
- Implementar una interfaz gráfica de usuario (GUI) que permita una interacción clara, intuitiva y funcional con el sistema.
- Construir y conectar una base de datos que almacene de manera segura la información relacionada con productos, ventas, clientes y demás operaciones.

4. Análisis de Arquitectura y Diseño

4.1. Diagrama de Clases y Relaciones

El Diagrama de Clases UML completo (Figura 1) ilustra la **estructura estática** de todo el sistema, mostrando las 12 clases, sus atributos, métodos y las relaciones que las vinculan.

Figura 1: Diagrama UML Completo del Sistema de Farmacia



Las relaciones principales son:

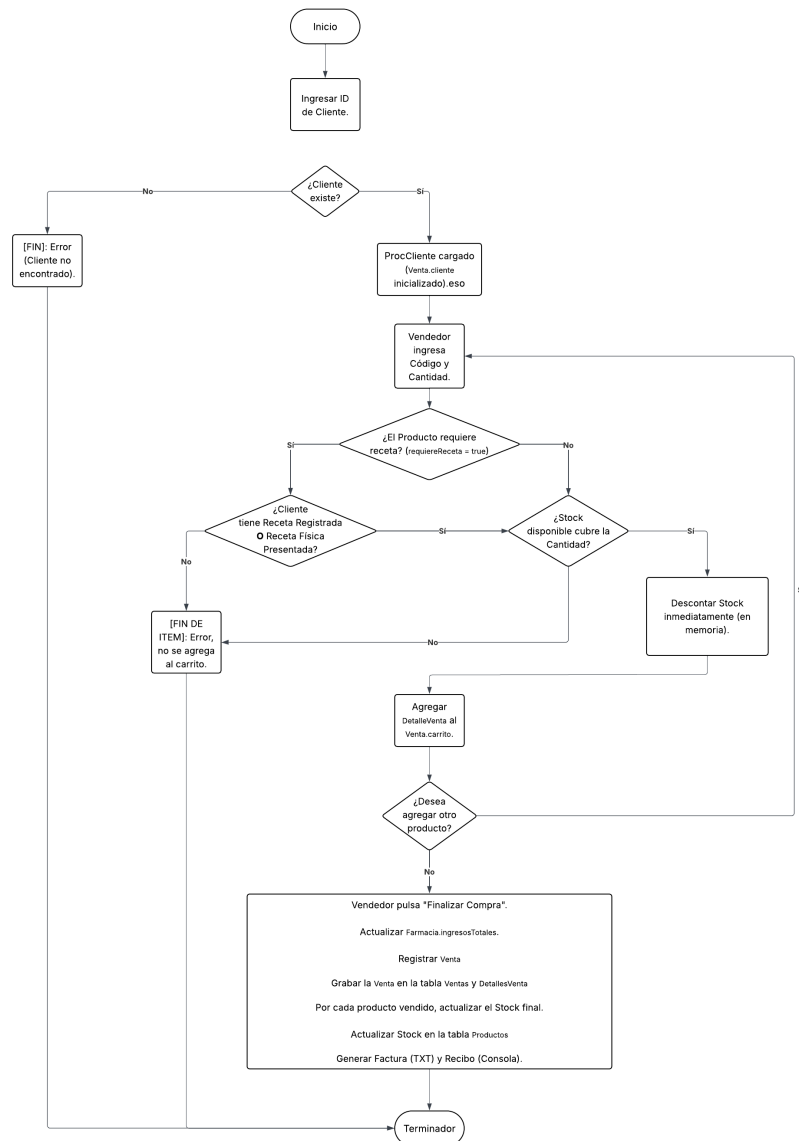
- **Agregación:** La clase **Farmacia** contiene listas de **Producto**, **Cliente** y **Venta**.
- **Composición:** Una **Venta** se compone de uno o más **DetalleVentas**.
- **Dependencia:** La clase **Farmacia** depende de **FarmaciaDAO** y **FarmaciaDAO** depende de **Conexion** para manejar la Base de Datos.

4.2. Diagrama de Flujo del Proceso de Venta

El Diagrama de Flujo (Figura 2) representa la lógica del **Proceso de Venta**, que es la funcionalidad central del programa, desde la interfaz hasta el registro final en la base de datos.

Figura 2: Diagrama de Flujo del Proceso de Venta (Funcionamiento General)

Diagrama en blanco
Juan Lorenzo Barajas Moya | Diciembre 9, 2023



El flujo se centra en validaciones críticas: la existencia y validación del **Cliente**; y un ciclo de adición de productos donde el sistema verifica si el **Stock es suficiente** y si se

cumplen los **requisitos de Receta Médica**. Una vez que todas las validaciones pasan, se procede a registrar la venta y a actualizar el stock en la Base de Datos.

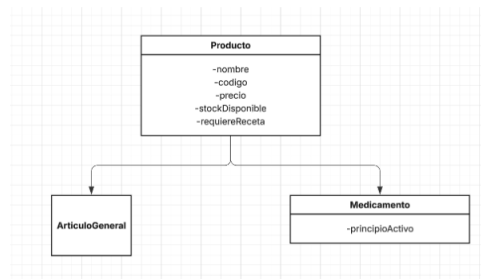
4.3. Jerarquía (Herencia)

La **Herencia** es un pilar fundamental de POO que permite que una clase (la subclase o clase hija) adquiera las propiedades (atributos) y comportamientos (métodos) de otra clase (la superclase o clase padre). Es crucial para manejar diferentes tipos de productos de manera eficiente, por eso lo organizamos de manera jerárquica.

4.3.1. Jerarquía de Productos

La superclase es **Producto**. Las clases hijas son **Medicamento** y **ArticuloGeneral**.

Figura 3: Jerarquía de Entidades: Productos



```
public abstract class Producto {
    // Definición de atributos y métodos abstractos/base
    // ...
}
```

Listing 1: Clase Producto: Constructor de la Superclase (Definición Base)

- **Clase ArticuloGeneral:** Extiende a **Producto** y hereda sus atributos.

```
public class ArticuloGeneral extends Producto {

    public ArticuloGeneral(String nombre, String codigo, double
    precio, int stockDisponible) {
        // Los artículos generales nunca requieren receta (false
    )

        super(nombre, codigo, precio, stockDisponible, false);
    }
}
```

Listing 2: Clase ArticuloGeneral: Constructor y Herencia

- **Clase Medicamento:** También se extiende a **Producto**. La única diferencia entre estas dos subclases es que **Medicamento** tiene un atributo específico que no tiene **ArticuloGeneral**: **principioActivo**. Esto es por la misma naturaleza de los medicamentos.

```
public class Medicamento extends Producto {
    private String principioActivo;
    public Medicamento(String nombre, String codigo, double precio, int
        stockDisponible, boolean requiereReceta, String principioActivo) {
    super(nombre, codigo, precio, stockDisponible, requiereReceta);
    this.principioActivo = principioActivo;
    }
}
```

Listing 3: Clase Medicamento: Herencia y Atributo Único

4.3.2. Jerarquía de Personas

La clase base **Persona** define la identidad básica que es común a cualquier individuo en el sistema.

Tanto **Cliente** como **Empleado** extienden a **Persona** usando la palabra clave **extends**. Esto significa que cada **Cliente** y cada **Empleado** automáticamente tienen **nombre** e **id**.

Clase Derivada	Extiende a	Atributo Único Adquirido	Utilidad de la Herencia
Cliente	Persona	tieneRecetaRegistrada	Un cliente necesita una identidad básica
Empleado	Persona	usuario, password	Un empleado necesita una identidad básica

Cuadro 1: Uso de Herencia en la Jerarquía de Personas

```
public class Cliente extends Persona {
    private boolean tieneRecetaRegistrada;
    public Cliente(String nombre, String id, boolean tieneRecetaRegistrada) {
    super(nombre, id);
    this.tieneRecetaRegistrada = tieneRecetaRegistrada;
    }
    public boolean isTieneRecetaRegistrada() { return tieneRecetaRegistrada; }
}
```

Listing 4: Clase Cliente: Herencia de Persona y Atributo Específico

4.4. Sobreescritura (Polimorfismo)

El polimorfismo por **Overriding** permite que objetos de diferentes clases hijas respondan de manera distinta al mismo método, permitiendo a la subclase modificar o especializar dicho comportamiento.

4.4.1. Aplicación en la Jerarquía de Personas: mostrarInfo()

El método `mostrarInfo()` se define primero en la clase base `Persona` para mostrar los datos de identidad comunes y luego se especializa en `Cliente`.

Clase	Comportamiento Especializado
Persona (Base)	Muestra solo el nombre y el id.
Cliente (Especializada)	Llama a la versión del padre (<code>super.mostrarInfo()</code>) y añade la información c

Cuadro 2: Polimorfismo en la Jerarquía de Personas

```
@Override //Polimorfismo
public void mostrarInfo() {
    super.mostrarInfo() ;
    String tiene = this.tieneRecetaRegistrada ? "S " : "No";
    System.out.println("Tiene Receta Registrada: " + tiene);
}
```

Listing 5: Sobreescritura en Cliente para mostrar Receta Registrada

4.4.2. Aplicación en la Jerarquía de Productos: mostrarInfo()

Aquí la sobreescritura tiene un impacto directo en la presentación del inventario.

Clase	Comportamiento Especializado
Producto (Base)	Muestra el código, nombre, precio, stock y si <code>requiereReceta</code> .
Medicamento (Especializada)	Llama a la versión del padre (<code>super.mostrarInfo()</code>) y añade el campo

Cuadro 3: Polimorfismo en la Jerarquía de Productos

```
@Override // Polimorfismo
public void mostrarInfo() {
    super.mostrarInfo();
    System.out.printf(" | Tipo: Medicamento | Principio Activo: %s\n", this.
        principioActivo);
}
```

Listing 6: Sobreescritura en Medicamento para añadir Principio Activo

4.5. Clases Abstractas y Encapsulamiento

4.5.1. Clase Abstracta (Producto): Forzando el Diseño

El uso de la clase `Producto` como **Clase Abstracta** (`public abstract class Producto`) es una técnica de diseño que impone reglas en la estructura del código.

1. **No se pueden crear objetos genéricos:** Al ser abstracta, es *imposible* en Java crear directamente un objeto genérico con la instrucción `new Producto(...)`. Esto obliga al desarrollador a clasificar cada nuevo ítem del inventario como `Medicamento` o `ArticuloGeneral`.
2. **Define un Contrato Mínimo:** Aunque no permite crear objetos, sí define qué atributos y métodos *deben* tener todas sus clases hijas. Esto garantiza la uniformidad en el inventario.

4.5.2. Encapsulamiento: Protegiendo el Corazón del Inventario

El ****Encapsulamiento**** es la práctica de ocultar los datos internos de un objeto y solo permitir el acceso a ellos a través de métodos controlados (getters y setters). Para un sistema de inventario, esto es vital para evitar errores de conteo.

1. **Protección de Atributos:** Los atributos esenciales, como el `stockDisponible` en `Producto.java`, se declaran como `protected` o `private`. Ninguna otra clase, ni siquiera la GUI, puede cambiar su valor directamente.

2. **Control de Comportamiento:** El único camino para alterar el stock es a través de métodos diseñados específicamente para ello, como `vender(cantidad)` o `agregarStock(cantidad)`. Estos métodos son los que contienen la lógica de protección.

```
public boolean vender(int cantidad) {
    if (this.stockDisponible >= cantidad) {
        // La validaci n ocurre antes de la modificaci n
        this.stockDisponible -= cantidad;
        return true; // Venta exitosa
    } else {
        // La manipulaci n directa est  bloqueada. El m todo fuerza
        // a verificar la condici n antes de modificar el atributo
        privado.
        return false; // Stock insuficiente , dato protegido
    }
}
```

Listing 7: Ejemplo de Encapsulamiento: El método vender() en Producto.java

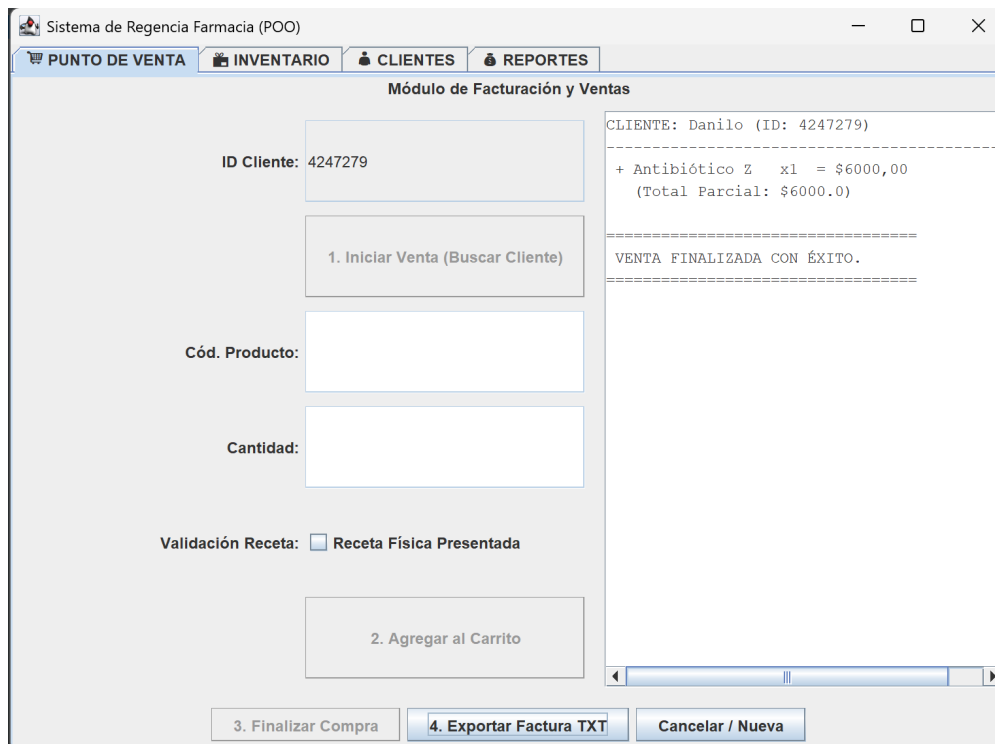
Este ejemplo ilustra el principio de **Encapsulamiento Inverso**: si la venta falla (`stockDisponible < cantidad`), el método devuelve `false`, manteniendo el valor del stock inalterado y protegiendo el dato de quedar en un estado negativo o incorrecto. Esta técnica es fundamental para mantener la ****Integridad del Inventario****.

5. Implementación de Interfaces

5.1. Interfaz Gráfica de Usuario (GUI)

El módulo `MenuPrincipal.java`, construido con la librería **Java Swing**, es el punto central de interacción para el vendedor. Su diseño sigue el principio de la **Separación de Responsabilidades**: la GUI solo se encarga de mostrar datos y capturar las acciones del usuario, mientras que la clase `Farmacia` maneja toda la lógica de negocio (validaciones, cálculos).

Figura 4: Captura del Módulo de Ventas de la GUI



5.1.1. Arquitectura y Acoplamiento Mínimo

La clase `MenuPrincipal` establece una relación de **Asociación** con la clase `Farmacia` en su constructor. Esta asociación es la base de la arquitectura de la GUI.

```
public class MenuPrincipal extends JFrame {  
    private Farmacia farmacia; // Referencia a la capa de negocio  
    private JTabbedPane tabbedPane;  
  
    public MenuPrincipal(Farmacia farmacia) {
```

```

        this.farmacia = farmacia; // Inyección de dependencia (
            Asociación 1:1)
        setTitle("Sistema de Regencia Farmacia");
        // ... configuración de pestañas (JTabbedPane) ...
    }
}

```

Listing 8: Inicialización y Asociación de MenuPrincipal con la Capa de Negocio (farmacia.java)

5.1.2. Delegación de la Lógica de Venta

El funcionamiento de la GUI se basa en la **Delegación**. Cuando el vendedor pulsa el botón “Agregar al Carrito”, la GUI toma el código y la cantidad, y le pasa la responsabilidad de la validación a la clase Farmacia.

```

// Ejemplo dentro de MenuPrincipal
btnAgregar.addActionListener(e -> {
    try {
        String codigo = txtCodigo.getText();
        int cantidad = Integer.parseInt(txtCantidad.getText());

        // La GUI DELEGA la validación a la capa de Farmacia.
        boolean exito = farmacia.agregarProductoACarritoTemporal(codigo,
            cantidad);

        if (exito) {
            JOptionPane.showMessageDialog(this, "Producto agregado.", "
                xito ", JOptionPane.INFORMATION_MESSAGE);
            // actualizarCarrito
        } else {
            // ... mostrar error de Stock o Receta
        }
    } catch (NumberFormatException ex) {
        // ... manejo de error
    }
});

```

Listing 9: Ejemplo de Captura de Evento y Delegación de la Lógica de Venta

Este patrón de **Delegación** garantiza que la capa de presentación (GUI) esté mínimamente acoplada a la lógica de negocio.

5.1.3. Sincronización de Vistas mediante JTable

Para mostrar la lista de productos y el carrito de compras, se usa el componente `JTable`. Cada vez que el inventario se modifica, la GUI debe sincronizarse con la información más reciente de la clase `Farmacia`.

```
public void actualizarTablaInventario() {
    DefaultTableModel model = (DefaultTableModel) tablaInventario.getModel
        ();
    model.setRowCount(0); // Limpia filas

    // Recorreremos la lista de Productos de la capa de negocio
    for (Producto p : farmacia.getProductos()) {
        Object[] row = new Object[]{
            p.getCodigo(),
            p.getNombre(),
            p.getPrecio(),
            p.getStock(),
            (p instanceof Medicamento ? "Medicamento" : "Art ículo General
                ")
        };
        model.addRow(row);
    }
}
```

Listing 10: Método de Sincronización de Datos y Polimorfismo en la GUI (actualizarTablaInventario)

La tabla de inventario demuestra el **Polimorfismo en la GUI**, ya que itera sobre la lista de `Productos`, que contienen instancias de clases hijas (`Medicamento` y `ArticuloGeneral`).

5.2. Base de Datos (B.D.)

La base de datos utilizada fue **SQLite**, y se gestiona mediante el patrón **Data Access Object (DAO)**.

5.2.1. Conexión y Driver JDBC

La base de datos que se usó para este proyecto fue SQLite por su baja complejidad. Para establecer la comunicación entre la aplicación JAVA y SQLite, fue necesario utilizar un driver JDBC.

Figura 5: Driver JDBC de SQLite



Se establece la clase `Conexion.java` como el componente clave para la gestión de datos.

```
public class Conexion {

    private static final String URL = "jdbc:sqlite:bdfarmacia.db";

    public Connection establecerConexion() {
        Connection connection = null;
        try {

            Class.forName("org.sqlite.JDBC");
            connection = DriverManager.getConnection(URL);

        } catch (ClassNotFoundException e) {
            JOptionPane.showMessageDialog(null, "Error: No se
encontr el driver JDBC de SQLite.", "Error de Driver", JOptionPane.
ERROR_MESSAGE);
            System.err.println("Driver not found: " + e.getMessage()
);
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al conectar
con la base de datos.", "Error de Conexi n", JOptionPane.ERROR_MESSAGE
);
        }
    }
}
```

```

        System.err.println("SQL Exception: " + e.getMessage());
    }
    return connection;
}
// ... m todos para cerrar la conexi n ...
}

```

Listing 11: Conexion.java: Método para establecer la conexión

5.2.2. Capa DAO: Mantenimiento y CRUD

La clase `FarmaciaDAO.java` interactúa directamente con la base de datos, funcionando como la capa CRUD (Crear, Leer, Actualizar, Borrar).

- **Creación de Tablas:** El método inicial `crearTablasSiNoExisten()` asegura que la estructura de la B.D. esté presente.
- **Registro (CREATE):** El método `registrarCliente` registra los atributos del objeto `Cliente` en la Base de Datos.
- **Lectura (READ):** El método `obtenerTodosLosClientes` ejecuta las sentencias `SELECT` para recuperar el estado completo de los clientes.
- **Actualización (UPDATE):** El método `actualizarStock` ejecuta la sentencia `UPDATE` para reflejar el descuento en el inventario.

```

public void registrarCliente(Cliente c) {
    Connection con = null;
    String sql = "INSERT OR REPLACE INTO Clientes (id, nombre,
        tieneRecetaRegistrada) VALUES (?, ?, ?)";
    // ...
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, c.getId());
        ps.setString(2, c.getNombre());
        ps.setBoolean(3, c.isTieneRecetaRegistrada());
        ps.executeUpdate();
    }
    // ...
}

```

Listing 12: FarmaciaDAO.java: Método para registrar un objeto Cliente (CREATE)

```
public boolean actualizarStock(String codigoProducto, int nuevoStock) {
    Connection con = null;
    String sql = "UPDATE Productos SET stockDisponible = ? WHERE codigo = ?";
    // ...
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, nuevoStock);
        ps.setString(2, codigoProducto);
        return ps.executeUpdate() > 0;
    }
    // ...
}
```

Listing 13: FarmaciaDAO.java: Método para actualizar el stock (UPDATE)

La operación DELETE del CRUD no se implementó debido a que no era esencial para enfocarse en la gestión de transacciones.

6. Conclusiones

Las conclusiones demuestran el cumplimiento de los objetivos planteados al inicio del proyecto, basándose en la arquitectura implementada:

- ****Relacionado al Objetivo de Desarrollar Módulos Centrales:**** Se implementó con éxito el módulo de ventas, integrando el carrito de compras, el registro de clientes y el control de inventario. La lógica del negocio garantiza que la venta solo se complete si el stock está disponible y si se cumplen los requisitos de receta médica para los medicamentos controlados.
- ****Relacionado al Objetivo de Implementar GUI:**** La Interfaz Gráfica de Usuario, desarrollada en Java Swing, cumplió su propósito al ofrecer un entorno de interacción claro e intuitivo. Se logró la separación de responsabilidades, donde la GUI solo actúa como controlador, delegando todas las validaciones complejas a la capa de negocio (`Farmacia.java`).
- ****Relacionado al Objetivo de Construir Capa de Persistencia:**** Se estableció una capa de persistencia funcional con SQLite y el patrón DAO. Esto garantiza que todos los registros de clientes, productos y el historial de ventas se almacenen de forma segura y persistan tras el cierre del programa.
- ****Relacionado al Objetivo de Demostrar Pilares POO:**** El diseño del sistema demuestra una aplicación robusta de los pilares POO. La ****Herencia**** permitió gestionar diferentes tipos de productos de forma modular, el ****Polimorfismo**** simplificó la presentación de información, y el ****Encapsulamiento**** garantizó la integridad crítica del inventario, evitando errores en el control de stock.